

Lab 8: Disk Management: Disk Scheduling Algorithms

The simulator allows you to generate random data for illustration. Use random data.

Where necessary copy and paste images of the data in question throughout your document to illustrate your answers.

If necessary re-run the program or generate new data but make sure to show the new data in question.

General Questions

1. Fully explain the meaning of the following terms **used by the simulation**:

- a. **Disk Track Request**

A disk track request refers to a request made by a process or user to access a specific track (location) on the disk. The disk is divided into concentric circles called tracks, and each track holds data. A disk track request specifies the track (or cylinder) number that the disk arm (read/write head) must move to in order to read or write data.

- b. **Disk Scheduling Algorithm**

A disk scheduling algorithm is a method used to determine the order in which disk track requests are processed. The purpose of disk scheduling is to optimize the use of the disk, reducing the time the disk arm takes to move, and ensuring that requests are handled efficiently. Examples of disk scheduling algorithms include First Come First Serve (FCFS), Shortest Seek Time First (SSTF), Scan, C-SCAN, Look, and C-Look.

- c. **Number of tracks crossed. What does the value of the 'number of tracks crossed' indicate?**

The number of tracks crossed refers to the number of tracks the disk arm moves across when servicing a series of disk track requests. This is a measure of the total movement of the disk arm. A lower number of tracks crossed indicates more efficient disk scheduling because the arm doesn't have to move excessively.

What does the value of 'number of tracks crossed' indicate? The value of the number of tracks crossed indicates the efficiency of the disk scheduling algorithm. A smaller value means the algorithm is performing well in minimizing disk arm movement, reducing latency, and increasing disk throughput. Higher values indicate inefficient scheduling, resulting in more disk arm movements and longer response times.

d. Scan direction

Scan direction refers to the direction in which the disk arm moves while processing requests in certain disk scheduling algorithms like the **SCAN** or **C-SCAN** algorithms. For example, in the **SCAN** algorithm, the disk arm moves in one direction (e.g., from the outermost track to the innermost track), servicing requests along the way. Once the arm reaches the end of the disk, it reverses direction and starts processing requests in the opposite direction.

- e. Value of 'N' in N-Step-Scan:** In the **N-Step-Scan** algorithm, 'N' represents the number of track requests that will be processed in each step of the scan. The disk arm processes these requests in one direction and then reverses direction once all requests in the current step are handled. The value of 'N' helps divide the requests into smaller batches, which can be handled more efficiently by scanning a subset of tracks at a time.

- f. Disk Arm:** The disk arm refers to the mechanical component that moves the read/write head across the surface of the disk. The disk arm is responsible for positioning the head at the correct track to read or write data. The arm moves horizontally across the disk's tracks to access different areas on the disk.

- g. Starting Track of Disk R/W Head:** The starting track refers to the initial position of the read/write head (disk arm) at the beginning of the scheduling process. The simulation typically sets an initial track where the head begins, and the disk scheduling algorithm will calculate the most efficient order of processing requests from this starting point.

Method Specific Exercises

The purpose of the exercises is to investigate the properties and behaviour of the disk scheduling algorithms discussed in class. When explaining the results of each investigation **always relate your explanations to how the specific algorithms at work behave using examples.**

For example, if using CScan, then explain why the tracks are visited in the order shown in terms of how the CScan algorithm made those decisions.

1. Run each algorithm simulation and for each one:
 - a. Use the snipping tool to copy and paste the data used and the output generated into your document.
 - b. Fully **explain why** the relevant scheduling algorithm made the choices it did in visiting the tracks in the order shown.

2. SSTF allows 'starvation' but the others do not. Explain what 'starvation' means. Use the facility to 'add data while tracks are being processed' in the simulation program to investigate starvation. Are the new track requests able to jump the queue? Why is that? Try to add new requests during the run of the FIFO algorithm to see if there is a difference. Record your observations.
3. The Scan, CScan, Look & CLook algorithms all suffer from 'arm stickiness'. Explain what 'arm stickiness' means. Use the facility to 'add data while tracks are being processed' in the simulation program to investigate 'arm stickiness'. Are the new track requests able to jump the queue? Why is that? Try to add new requests during the run of the FScan algorithm to see if there is a difference. Record your observations.
4. Show the results obtained for a larger and smaller value of N in N-Step-Scan. Compare and contrast these results and explain the differences.
5. Show the results obtained for the same value of N in N-Step-Scan & N-Step-Look. Compare and contrast these results and explain the differences.

First In, First Out FIFO Disk Scheduling Algorithm

Default Data, don't add data while tracks are being processed

```
***** First In, First Out *****  
  
Will you want to add data while tracks are being processed (y/[n])? n  
In the FIFO algorithm simulation the following track requests were input  
(in order of arrival): [98, 183, 37, 122, 14, 124, 65, 67]  
and at the beginning of the simulation the r/w head was initially at track 53  
Track requests were processed in the following order:  [98, 183, 37, 122, 14, 124, 65, 67]  
Numbers of tracks crossed for each request:  
From 53 to 98 : 45  
From 98 to 183 : 85  
From 183 to 37 : 146  
From 37 to 122 : 85  
From 122 to 14 : 108  
From 14 to 124 : 110  
From 124 to 65 : 59  
From 65 to 67 : 2  
  
Total number of tracks crossed: 640  
  
Press <ENTER> to continue  
  
Assuming the disk tracks are numbered from 0 to 199 and the disk arm is initially at track 53  
  
And that CScan & CLook only process requests on the way up toward 199  
  
8 original disk track requests as follows:  
[98, 183, 37, 122, 14, 124, 65, 67]
```

When a track is requested, it goes to the back of the queue. The Track requests are processed in the order in which they are requested. The first one that was requested is the first one to be processed. Track 98 was requested first; the track is at 53 and it then goes to 98 crossing 45 tracks. 183 is the next track in the queue so the track will cross 85 tracks and go there next. The next is the queue is 37 and even though 122 is closer to where the track currently is it follows the order in which the requests

arrivals so it will cover 145 tracks and go down to 37. It then goes back up to 122. Once again back down to 14. The track then goes back up to 124. It goes to 65 and final it covers 2 tracks to 67. It covered a total of 640 tracks going from track to track in the order that the requests arrived.

Shortest Seek Time First (SSTF) Disk Scheduling Algorithm

Default Data, don't add data while tracks are being processed

```
***** Shortest Seek Time First *****  
Will you want to add data while tracks are being processed (y/[n])? n  
In the SSTF algorithm simulation the following track requests were input  
(in order of arrival): [98, 183, 37, 122, 14, 124, 65, 67]  
and at the beginning of the simulation the r/w head was initially at track 53  
Track requests were processed in the following order: [65, 67, 37, 14, 98, 122, 124, 183]  
Numbers of tracks crossed for each request:  
From 53 to 65 : 12  
From 65 to 67 : 2  
From 67 to 37 : 30  
From 37 to 14 : 23  
From 14 to 98 : 84  
From 98 to 122 : 24  
From 122 to 124 : 2  
From 124 to 183 : 59  
  
Total number of tracks crossed: 236  
  
Press <ENTER> to continue  
  
Assuming the disk tracks are numbered from 0 to 199 and the disk arm is initially at track 53  
  
And that CScan & CLook only process requests on the way up toward 199  
  
8 original disk track requests as follows:  
[98, 183, 37, 122, 14, 124, 65, 67]
```

The process that is nearest to the track is the next one that will be processed. The track is at 53 and the closest track to that track is 65 so that track will be processed next. It then goes to 67. The next closest track is 37, then it goes to 14. The track then must go up to 98 that the next closest track. It then goes to 122, then 124 and finally it goes to 183. By going to the next closest track, it covers 236 tracks.

Scan Disk Scheduling Algorithm

Default Data, scan direction 0 to 199. Don't add data while tracks are being processed

```
***** scan Algorithm *****

Choose Scan Direction

1:    From 0 to 199
2:    From 199 to 0
Which scanning direction (1 or 2)?1
Will you want to add data while tracks are being processed (y/[n])? n
In the Scan algorithm simulation the following track requests were input
(in order of arrival): [98, 183, 37, 122, 14, 124, 65, 67]
and at the beginning of the simulation the r/w head was initially at track 53
Track requests were processed in the following order:  [65, 67, 98, 122,
124, 183, 199, 37, 14, 0]
Numbers of tracks crossed for each request:
From 53 to 65 : 12
From 65 to 67 : 2
From 67 to 98 : 31
From 98 to 122 : 24
From 122 to 124 : 2
From 124 to 183 : 59
From 183 to 199 : 16
From 199 to 37 : 162
From 37 to 14 : 23
From 14 to 0 : 14

Total number of tracks crossed: 345

Press <ENTER> to continue

Assuming the disk tracks are numbered from 0 to 199 and the disk arm is initially at track 53

And that CScan & CLook only process requests on the way up toward 199

8 original disk track requests as follows:
[98, 183, 37, 122, 14, 124, 65, 67]
```

The track is scanning from 0 to 199 and it will process the tracks as it passes through. The track was originally at track 53, it continues up the Disk towards 199 so the first track it will process is 65, It will then process 67, then continue up to 98. Process 122 and then 124. It will keep processing the tracks going up to 199 so next is 183 and then finally it reaches 199. The track direction will now change, and it will pick up tracks on its way down to 0. It goes from 199 to 37 covering 162 tracks. It will then go to 14 and finally it will end at track 0. It covered a total of 345 tracks.

Look Disk Scheduling Algorithm

Default Data, don't add data while tracks are being processed

```
***** look Algorithm *****  
  
Choose Scan Direction  
  
1:    From 0 to 199  
2:    From 199 to 0  
Which scanning direction (1 or 2)?1  
Will you want to add data while tracks are being processed (y/[n])? n  
In the Look algorithm simulation the following track requests were input (in  
order of arrival): [98, 183, 37, 122, 14, 124, 65, 67]  
and at the beginning of the simulation the r/w head was initially at track 5  
3  
Track requests were processed in the following order:  [65, 67, 98, 122, 12  
4, 183, 37, 14]  
Numbers of tracks crossed for each request:  
From 53 to 65 : 12  
From 65 to 67 : 2  
From 67 to 98 : 31  
From 98 to 122 : 24  
From 122 to 124 : 2  
From 124 to 183 : 59  
From 183 to 37 : 146  
From 37 to 14 : 23  
  
Total number of tracks crossed: 299  
  
Press <ENTER> to continue  
  
Assuming the disk tracks are numbered from 0 to 199 and the disk arm is init  
ially at track 53  
  
And that CScan & CLook only process requests on the way up toward 199  
  
8 original disk track requests as follows:  
[98, 183, 37, 122, 14, 124, 65, 67]
```

C-Scan Disk Scheduling Algorithm

Default Data, don't add data while tracks are being processed

- Complete up to C-Scan and then go back and do the same thing but this time add data while tracks are being processed to see how it would change the output, starvation interrupts etc.
 - Answer the questions at the start of the document.
-
- **First Come First Serve (FCFS):**
 - FCFS processes requests in the order they are received, without considering the current position of the disk arm. This can lead to inefficient movement of the disk arm, as it may travel to far-away tracks instead of the nearest ones.
 - Example: If the requests are at tracks 20, 50, 30, and 10, and the starting position is track 25, the disk arm will move to track 20 first, then 50, then 30, and finally 10.
 - **Shortest Seek Time First (SSTF):**
 - SSTF selects the track request closest to the current position of the disk arm. It minimizes the time the arm spends moving by choosing the nearest track each time.
 - Example: If the starting position is track 25, and the requests are at tracks 20, 50, 30, and 10, SSTF will first move to track 20, then 30, then 10, and finally 50.
 - **SCAN (Elevator Algorithm):**
 - The SCAN algorithm moves the disk arm in one direction, servicing requests as it goes, and then reverses direction when it reaches the end of the disk.
 - Example: If the arm starts at track 25 and requests are at tracks 20, 50, 30, and 10, the arm will first move towards the nearest end (say the lower tracks), processing requests along the way. Once it reaches the end of the disk, it reverses direction and processes requests in the opposite direction.
 - **C-SCAN (Circular SCAN):**
 - C-SCAN is similar to SCAN but instead of reversing direction, once the arm reaches the end, it jumps back to the opposite end of the disk and continues processing requests.
 - Example: If the arm starts at track 25 and requests are at tracks 20, 50, 30, and 10, the arm will move towards the higher tracks, and once it reaches the end, it jumps back to the beginning of the disk and continues processing requests.

SSTF and Starvation

- **Starvation** occurs when a track request is continuously bypassed because there are other requests closer to the current position of the disk arm. This happens in algorithms like **SSTF**, where requests further away may not be processed if there are always nearer requests.
- **Investigation with new track requests:** In SSTF, if you add new track requests during the simulation, the new requests may "jump the queue" if they are closer to the current position of the disk arm. This leads to **starvation** for distant requests that are never serviced.
 - **FIFO (First In, First Out)** does not allow new requests to jump the queue, so requests are processed in the order they are received, regardless of the current position of the disk arm.

Scan, C-Scan, Look, C-Look and 'Arm Stickiness'

- **Arm Stickiness** refers to the situation where the disk arm repeatedly processes requests in one direction and "sticks" to that direction due to the nature of the scheduling algorithm. This can lead to inefficient movement as the arm may repeatedly service requests in one direction, ignoring pending requests in the opposite direction.
- **Investigating Arm Stickiness:** When using algorithms like **SCAN**, **C-SCAN**, **LOOK**, or **C-LOOK**, adding new track requests during processing can cause the disk arm to get "stuck" in one direction. For example, in **SCAN**, the arm may continue moving in one direction even when there are new requests in the opposite direction, leading to a delay in processing those requests.
 - **FScan** may behave differently, as it processes requests in steps, and adding new requests may cause less "stickiness" since it divides the requests into batches.